

HW03

News Recommendation Prediction

313581038 智能系統碩一 蒲品憶

1. (10%) Describe how you solve this problem. Details include preprocessing, embeddings, model selection, hyperparameters should be provided

- **預處理:**

- 讀取 tsv 檔案: `pd.read_csv(file_path, sep='\t', header=None, names=[欄位名稱 1, 欄位名稱 2...])`
- 缺失數值: 將空值填充為空字符串\
- News 資料: 將訓練集和測試集的新聞資料合併, 並去除重複的新聞資料, 為後續生成 [新聞字典] 做準備

- **嵌入向量:**

- 模型使用: 使用 SentenceTransformer model, `model_name = BAAI/bge-base-en-v1.5` 作為嵌入模型, 適合用於生成文本的嵌入表示。該模型的嵌入維度為 768, 這意味著每篇新聞都會被轉換為一個 768 維的向量表示。

```
SentenceTransformer(model_name, device=DEVICE)
embeddings = model.encode(batch_texts, convert_to_tensor=False,
show_progress_bar=False)
```

- News 資料: 將剛才合併後的新聞資料, 利用嵌入模型轉換成嵌入向量, 並且建立新聞字典, 方便使用者 behavior 資料及, 可以直接根據新聞 ID 取得嵌入向量。
創建前處理將欄位串接, 使用欄位: title, abstract, category, subcategory, 加入 " [SEP] " 作為字串連接符

```
news_embeddings_dict = {}
news_df['combined_text'] = news_df['title'].astype(str) + " [SEP] " + \
news_df['abstract'].astype(str) + " [SEP] " + \
news_df['category'].astype(str) + " [SEP] " + \
news_df['subcategory'].astype(str)
texts_to_encode = news_df['combined_text'].tolist()
news_ids = news_df['news_id'].tolist()
logger.info(f"共有 {len(texts_to_encode)} 篇新聞待編碼。")
```

- Behavior 資料集: 透過新聞 ID 獲取所需要的嵌入向量，格式如下:

➤ Train

```
processed_data.append({
    'user_history_embedding': user_history_embedding,
    'candidate_news_embedding': candidate_news_embedding,
    'label': clicked_label
})
```

➤ Test/valid

```
impression_samples_for_this_user.append({
    'user_history_embedding': user_history_embedding,
    'candidate_news_embedding': candidate_news_embedding,
})
```

測試集需要有 id 作為輸出結果排序

```
processed_data.append({
    'id': submission_id,
    'samples': impression_samples_for_this_user,
})
```

- 注意: 這邊完全不使用資料集裡面給的 entity_embedding.vec，全部嵌入向量轉換都直接由 SentenceTransformer model 完成
- 訓練模型:**
 - 模型: 使用 Feed-Forward Network (FFN) 模型來進行推薦預測。這個模型接受兩個嵌入向量作為輸入：一個是用戶的歷史行為嵌入，另一個是候選新聞的嵌入。這兩個嵌入被拼接起來作為神經網絡的輸入。
 - 結構: 包含多個隱藏層，每層後面都接有批量正規化 (BatchNorm)、ReLU 激活函數和 Dropout 層來防止過擬合。模型結構如下:

```
(network): Sequential(
  (0): Linear(in_features=1536, out_features=768, bias=True)
  (1): BatchNorm1d(768, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (2): ReLU()
  (3): Dropout(p=0.42706370758034395, inplace=False)
  (4): Linear(in_features=768, out_features=384, bias=True)
  (5): BatchNorm1d(384, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (6): ReLU()
  (7): Dropout(p=0.42706370758034395, inplace=False)
  (8): Linear(in_features=384, out_features=128, bias=True)
  (9): BatchNorm1d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (10): ReLU()
  (11): Dropout(p=0.42706370758034395, inplace=False)
  (12): Linear(in_features=128, out_features=1, bias=True)
)
```

- 超參數:**
 - 優化: 這邊使用 Optuna 進行超參數優化，以下是學習率、批次大小、隱藏層維度、Dropout 率，在優化中設定的範圍。

```
# 讓 Optuna 建議超參數
lr = trial.suggest_float("lr", 1e-5, 1e-3, log=True)
batch_size = trial.suggest_categorical("batch_size", [256, 512, 1024]) # 可再增加 2048 選項
dropout_rate = trial.suggest_float("dropout_rate", 0.1, 0.5)

# 建議隱藏層配置
hidden_dims_options = [
    "config1": [512, 256],
    "config2": [512, 256, 128],
    "config3": [768, 384, 128],
    "config4": [1024, 512, 256],
    "config5": [256, 128]
```

2. (5%) Choose a variable (e.g. different model, different approach) excluding hyperparameters and compare their performance. (You cannot change ONLY the hyperparameters)

這邊我分別使用兩種不同大小的模型做編碼：

模型名稱	隱藏層維度	模型大小
BAAI/bge-small-en-v1.5	384	33.4M params
BAAI/bge-base-en-v1.5	768	109M params

在執行結果中，兩個模型的最佳驗證 AUC 分數如下：

BAAI/bge-small-en-v1.5	BAAI/bge-base-en-v1.5
最佳驗證 AUC: 0.7310	最佳驗證 AUC: 0.7344
 submission_deep_learning.csv Complete · 3d ago · BAAI/bge-small-en-v1.5 0.6973	 submission_deep_learning.csv Complete · 2d ago · BAAI/bge-base-en-v1.5 0.6983
NEWS_EMBEDDING_DIM: 384 LEARNING_RATE: 0.0001 BATCH_SIZE: 1024 HIDDEN_DIMS: [512, 256, 128] DROPOUT_RATE: 0.3 EPOCHS_RUN: 1	

性能差異分析：

從結果可以看到，BAAI/bge-base-en-v1.5 的驗證 AUC 分數高於 BAAI/bge-small-en-v1.5，儘管兩者的差距僅為 0.0034。這個小的差異表明，儘管兩個模型在性能上相差不多，但較大的模型 BAAI/bge-base-en-v1.5 在處理更複雜的數據模式時，仍然表現出一定的優勢。

為何 BAAI/bge-base-en-v1.5 優於 BAAI/bge-small-en-v1.5：

1. 更大的模型容量：

BAAI/bge-base-en-v1.5 擁有更多的參數（109M 參數），這意味著它擁有更多的能力來學習和捕捉數據中的複雜模式。較大的模型通常能夠學習到更多的細節，這有助於在複雜的預測任務中提供更高的性能。

2. 隱藏層維度的增大：

隱藏層維度從 384 增加到 768，這使得 BAAI/bge-base-en-v1.5 擁有更高的表示能力，可以處理更多的特徵和模式。這在處理需要細膩理解和長期依賴關係的文本數據時，往往會顯示出其優勢。

3. 模型的表現穩定性：

雖然兩個模型的差距不大，但較大的模型通常會在更長時間的訓練過程中顯示出更高的穩定性。BAAI/bge-base-en-v1.5 可能能夠更好地適應訓練數據，從而在驗證集上獲得略微更高的 AUC 分數。

4. 對細節的捕捉能力：

在文本分類和相似度計算等任務中，較大的模型可以捕捉到更多的語言特徵，這些特徵可能在較小的模型中無法完全顯現。因此，BAAI/bge-base-en-v1.5 的微小優勢可能來自於它在捕捉語言結構和語境上的精細差異。

總結：

雖然 BAAI/bge-small-en-v1.5 在速度和資源消耗方面具有一定的優勢，但 BAAI/bge-base-en-v1.5 在性能上明顯優於 BAAI/bge-small-en-v1.5，尤其是在複雜文本的理解和預測方面。隨著模型大小和隱藏層維度的增大，BAAI/bge-base-en-v1.5 能夠更有效地學習數據中的深層特徵，並在驗證集上取得略微更好的結果。因此，對於需要高性能預測的任務，BAAI/bge-base-en-v1.5 是更為理想的選擇。

3. (10%) Do error analysis or case study. Is there anything worth mentioning while checking the mispredicted data? Share with us. Anytime you try to make a conclusion about the data or model, you should provide concrete data example. [e.g. I think the model predicts poorly when ... (provide error examples)]

在檢查模型預測錯誤時，我發現有些錯誤預測存在顯著的模式，其中最主要的問題出現在以下幾個方面：

1. 特定新聞類別的預測不準確

我發現模型對於某些特定新聞類別的預測表現較差，特別是那些以技術類、科學類文章為主的新聞。在這些類別中，模型經常錯誤地將一些非熱門新聞標註為點擊量高的新聞，這可能與新聞的語言特徵或難以捕捉的專有名詞有關。

2. 長尾效應導致的錯誤預測

另一個常見的錯誤是在處理一些冷門或極少點擊的新聞時，模型的預測表現較差。這些冷門新聞的點擊量較低，但由於模型未能充分識別出這些新聞的特徵，可能會誤將它們歸類為有較高點擊量的新聞。

3. 用戶歷史行為的影響

模型的預測也受到用戶過去行為的強烈影響。在某些情況下，模型對於某些特定用戶的行為歷史記錄過於依賴，這導致當用戶的行為歷史較少或不具代表性時，模型的預測會出現較大的偏差。

總結：

基於這些錯誤預測，我認為模型的表現受到了以下因素的影響：

- 專有名詞和技術術語的挑戰：特定領域的文章可能包含複雜的術語，這對模型的預測

提出了挑戰。

- 長尾效應：模型對於低點擊量的新聞預測較為困難，尤其是冷門新聞。
- 用戶行為歷史的依賴：過度依賴用戶的歷史行為，忽略了用戶行為的動態變化，會影響預測的準確性。

這些錯誤分析結果可以幫助我們在未來進一步優化模型，進行更加精確的特徵選擇和訓練，以提升推薦的準確性和廣泛性